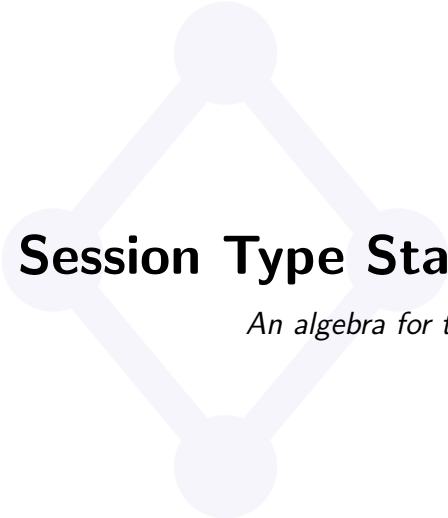




Session Type State Spaces Form **Lattices**

An algebra for the protocols we already write

MCP • Anthropic / JSON-RPC	A2A • Google / JSON-RPC	LSP Microsoft / JSON-RPC	gRPC • Google / Protobuf	Pub/Sub • Google / Protobuf
Signal • Signal Fdn. / Protobuf	HTTP • IETF / ABNF	SMTP • IETF / ABNF	OAuth 2.0 • IETF / ABNF	WebSocket • IETF / ABNF
IMAP IETF / ABNF	SIP IETF / ABNF	FTP IETF / ABNF	POP3 IETF / ABNF	NNTP IETF / ABNF
Kerberos • IETF / ASN.1	LDAP IETF / ASN.1	SNMP IETF / ASN.1	OCSP IETF / ASN.1	TLS • IETF / TLS Pres.
mTLS • IETF / TLS Pres.	DTLS IETF / TLS Pres.	SAML • OASIS / XML	SOAP W3C / XML	WS-Security OASIS / XML
XMPP IETF / XML	NFS IETF / XDR	Sun RPC IETF / XDR	SSH • IETF / Binary	MQTT • OASIS / Binary
Kafka • Apache / Binary	DNS • IETF / Binary	DNSSEC • IETF / Binary	QUIC IETF / Binary	AMQP OASIS / Binary
Redis Redis / Binary	RTP IETF / Binary	NTP IETF / Binary	DHCP IETF / Binary	STUN IETF / Binary

MCP • Anthropic / JSON-RPC	A2A • Google / JSON-RPC	LSP Microsoft / JSON-RPC	gRPC • Google / Protobuf	Pub/Sub • Google / Protobuf
Signal • Signal Fdn. / Protobuf	HTTP • IETF / ABNF	SMTP • IETF / ABNF	OAuth 2.0 • IETF / ABNF	WebSocket • IETF / ABNF
IMAP IETF / ABNF	SIP IETF / ABNF	FTP IETF / ABNF	POP3 IETF / ABNF	NNTP IETF / ABNF
Kerberos • IETF / ASN.1	LDAP IETF / ASN.1	SNMP IETF / ASN.1	OCSP IETF / ASN.1	TLS • IETF / TLS Pres.
mTLS • IETF / TLS Pres.	DTLS IETF / TLS Pres.	SAML • OASIS / XML	SOAP W3C / XML	WS-Security OASIS / XML
XMPP IETF / XML	NFS IETF / XDR	Sun RPC IETF / XDR	SSH • IETF / Binary	MQTT • OASIS / Binary
Kafka • Apache / Binary	DNS • IETF / Binary	DNSSEC • IETF / Binary	QUIC IETF / Binary	AMQP OASIS / Binary
Redis Redis / Binary	RTP IETF / Binary	NTP IETF / Binary	DHCP IETF / Binary	STUN IETF / Binary

Each is a protocol — each with its own **specification language**.

Each one can be read as a **session type**.

MCP • Anthropic / JSON-RPC	A2A • Google / JSON-RPC	LSP Microsoft / JSON-RPC	gRPC • Google / Protobuf	Pub/Sub • Google / Protobuf
Signal • Signal Fdn. / Protobuf	HTTP • IETF / ABNF	SMTP • IETF / ABNF	OAuth 2.0 • IETF / ABNF	WebSocket • IETF / ABNF
IMAP IETF / ABNF	SIP IETF / ABNF	FTP IETF / ABNF	POP3 IETF / ABNF	NNTP IETF / ABNF
Kerberos • IETF / ASN.1	LDAP IETF / ASN.1	SNMP IETF / ASN.1	OCSP IETF / ASN.1	TLS • IETF / TLS Pres.
mTLS • IETF / TLS Pres.	DTLS IETF / TLS Pres.	SAML • OASIS / XML	SOAP W3C / XML	WS-Security OASIS / XML
XMPP IETF / XML	FEWM IETF / XDR	STUN IETF / XDR	MQTT • OASIS / Binary	AMQP OASIS / Binary
Kafka • Apache / Binary	DNS • IETF / Binary	DNSSEC • IETF / Binary	QUIC IETF / Binary	STUN IETF / Binary
Redis Redis / Binary	RTP IETF / Binary	NTP IETF / Binary	DHCP IETF / Binary	STUN IETF / Binary

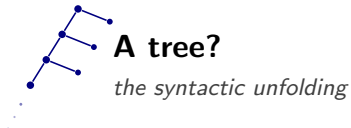
Each is a protocol — each with its own **specification language**.

Each one can be read as a **session type**.

We need an **algebra** for them.

What kind of mathematical object is a session type?

```
rec X . &{ hasNext:  $\oplus$ { TRUE: &{ next: X }, FALSE: end } }
```

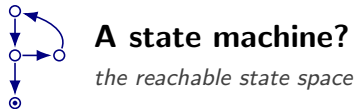


Pros

- + mirrors the type's syntax
- + nested choices explicit

Cons

- recursion unfolds **forever**
- the syntax, not the states



Pros

- + **finite**, even under recursion
- + the states actually reached

Cons

- dynamics only — what fires next
- is its reachability a *lattice*?

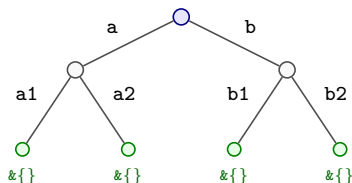
The state machine is the object — the finite states a protocol actually reaches. Reachability already orders them; we show that order is a **lattice** (meets, joins, $\parallel$ = product).

Where the lattice comes from: a shared bottom

$\&\{ a: \&\{ a1: \&\{\}, a2: \&\{\} \}, b: \&\{ b1: \&\{\}, b2: \&\{\} \} \}$

As a tree

(no end-identification)



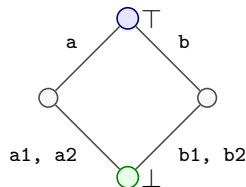
7 states, 4 distinct $\&\{\}$ leaves

As a lattice

(with end-identification)



share
 $\&\{\} = \text{end}$



4 states — a diamond

$\text{end} \equiv \&\{\}$: Vasconcelos, Gay, Ravara, Gesbert, Caldeira (2009)

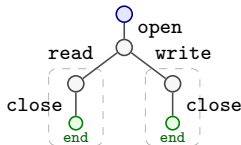
Gay, Vasconcelos, Ravara, Gesbert, Caldeira (POPL 2010)

Where the lattice comes from: named equations

As a tree

close inlined in every branch

```
&{open: &{read: &{close: end},  
  write: &{close: end}}}
```



&{close:end} duplicated

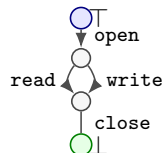


*name it
once as
Close*

As a lattice

Close named once — shared

```
&{open: &{read: Close, write: Close}},  
Close = &{close: end}
```



one shared Close — a lattice

Vasconcelos, Gay, Ravara, Gesbert, Caldeira (2009)

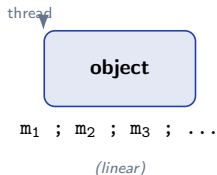
Gay, Vasconcelos, Ravara, Gesbert, Caldeira (POPL 2010)

Name the close protocol once: read and write reconverge on the shared Close — the tree becomes a lattice.

Where the lattice comes from: a parallel product

Prior works (2009 · 2010)

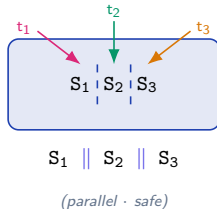
sharable object, linear access



- One thread per session
- Methods called **in sequence**
- Concurrency prevented by linearity

Insight

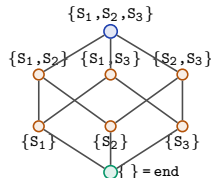
independent parallel services



- *Disjoint* protocols
- Files: read / seek; DBs: cursors; animals: breathe / walk / eat
- Safe by construction — **no interference**

... is already a lattice

subsets of $\{S_1, S_2, S_3\}$ under $\subseteq$



- Nodes: sub-sessions still active
- Boolean cube B_3 ($2^3=8$)
- Zoom in: each node is itself a session-type lattice

*Independent parallel services on one object — their product is a **lattice**.*

What an algebra buys us

A lattice is exactly **meet** $\sqcap$ and **join** $\sqcup$, total and unique — so each question has *one* answer.

Where did two runs last agree?	join $\sqcup$ · last common cause · rollback target
Where must they reconverge?	meet $\sqcap$ · synchronisation barrier
Order-independent merges?	modular? · Birkhoff: no N_5
Conflict-free concurrency?	distributive? · also no M_3
Compose two objects?	product · $\mathcal{L}(S_1) \times \mathcal{L}(S_2)$

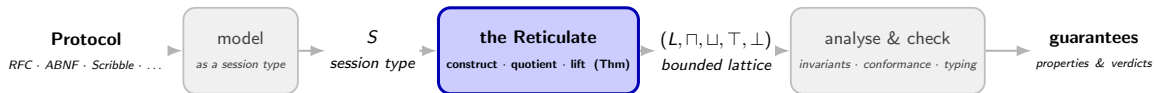
On the horizon — the power hidden in the lattice object, now turned on protocols:

Möbius function μ , characteristic & zeta polynomials, Whitney numbers, the order spectrum, $\#$ linear extensions, Birkhoff representation, Dedekind–MacNeille completion, congruence lattices.

All well-posed **because** it's a lattice — exactly what the three constructors bought us.

Methodology: From protocol to lattice

One methodology, **end to end**: any protocol becomes a session type, the **reticulate** lifts it to a **lattice**, and that lattice answers questions back about the protocol.



The core, realized three ways — and they agree on every well-formed protocol.

Theory

the *specification-level* construction

paper §2–3 · $\mathcal{L}(S)$, Def. 3

Lean 4

the same pipeline, *proven*

15 modules · zero sorry

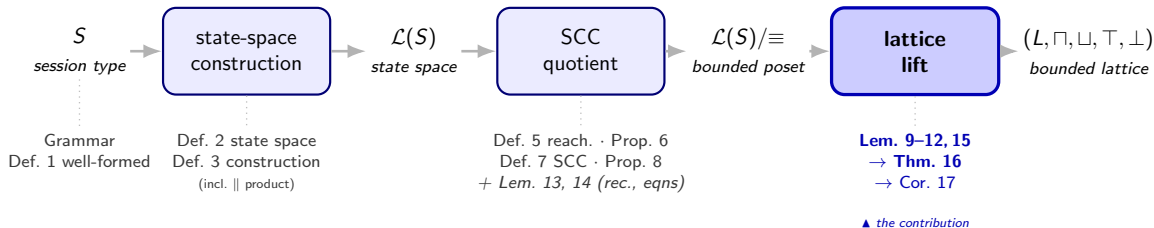
Implementation

the same pipeline, *executable*

RETICULATE (Py) · BICA REBORN (JVM)

Inside the Reticulate

The middle box, opened up: three deterministic steps — and the **last one is the contribution**.



Syntax: Grammar

Grammar (ICE 2026, n -ary):

S	$::=$	$\&\{m_1 : S_1, \dots, m_n : S_n\}$	(branch, $n \geq 0$)
		$\oplus\{\ell_1 : S_1, \dots, \ell_n : S_n\}$	(selection, $n \geq 0$)
		$S_1 \parallel \dots \parallel S_n$	(parallel, $n \geq 2$)
		$\mu X. S$	(recursion)
		X	(variable)
		end	(termination)
D	$::=$	$S, X_1 = S_1, \dots, X_n = S_n$	(declaration; named equations, incl. mutual rec.)

Blue marks the three syntactic sources of lattice structure: **end** (shared $\perp$) · named equations (shared sub-sessions) · $\parallel$ (product).

Syntax: Well-formedness (Def. 1)

Well-formed iff all five hold — each with a **counter-example** that violates it:

1	Closedness	no free variables	× $\&\{a:X\}$ (X free)
2	Contractiveness	guarded recursion	× $\mu X.X$ (unguarded)
3	Termination	a path to end from every state	× $\mu X.\&\{a:X\}$ (no end)
4	Parallel closedness	no outer-bound var free in a $\parallel$ -branch	× $\mu X.(\&\{a:X\} \parallel \mathbf{end})$
5	Distinct labels	labels pairwise distinct	× $\&\{a:\mathbf{end}, a:\mathbf{end}\}$

State space: construction (Def. 2 · 3)

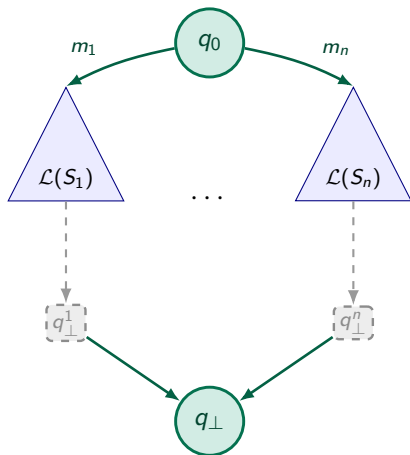
Def. 2 • State space. a tuple $(Q, \Sigma, \delta, q_{\top}, q_{\perp})$: finite states Q , action labels Σ , transitions $\delta \subseteq Q \times \Sigma \times Q$, initial $q_{\top}$, terminal $q_{\perp}$.

Def. 3 • Construction. $\mathcal{L}(S)$ by structural induction on closed well-formed S :

- ① $\mathcal{L}(\text{end}) = (\{q_0\}, \emptyset, \emptyset, q_0, q_0)$ — *a single state.*
- ② $\mathcal{L}(\&\{m_i:S_i\})$: glue children at q_0 , share terminal $q_{\perp}$ — *each m_i arrow into $\mathcal{L}(S_i)$.* ▷ pictured
- ③ $\mathcal{L}(\oplus\{\ell_i:S_i\}) = \mathcal{L}(\&\{\ell_i:S_i\})$ — *polarity-agnostic.*
- ④ $\mathcal{L}(\mu X. S') = \mathcal{L}(S')$ with X -placeholders redirected to $q'_{\top}$ — *back-edges.* ▷ pictured
- ⑤ $\mathcal{L}(D)$, $D = S$, $X_j=S_j$ — *glue each equation top; back/cross-edges.* ▷ pictured
- ⑥ $\mathcal{L}(S_1 \parallel \cdots \parallel S_n) = \mathcal{L}(S_1) \times \cdots \times \mathcal{L}(S_n)$ — *product.* ▷ pictured

State space: branch construction (Def. 3, branch case)

Add a fresh entry q_0 above the n children; drop each child bottom $q_{\perp}^i$ for one shared $q_{\perp}$.



$$\mathcal{L}(\&\{m_i:S_i\}) = (Q, \Sigma, \delta, \mathbf{q_0}, \mathbf{q_{\perp}})$$

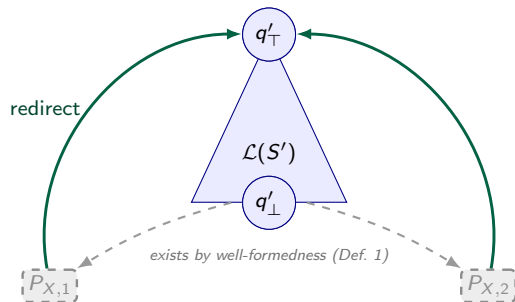
$$Q = \{\mathbf{q_0}\} \cup \bigcup_i (Q_i \setminus \{\mathbf{q_{\perp}^i}\}) \cup \{\mathbf{q_{\perp}}\}$$

$$\delta = \{(\mathbf{q_0}, m_i, \mathbf{q_{\perp}^i})\}_i \cup \bigcup_i \delta_i[\mathbf{q_{\perp}^i} := \mathbf{q_{\perp}}]$$

green = shared entry/terminal; grey = each child's old bottom, removed.

State space: recursion construction (Def. 3, recursion case)

For *each occurrence* of X in $\mathcal{L}(S')$, add a placeholder $P_{X,j}$; point each back to the top $q'_\top$ of $\mathcal{L}(S')$ — *new back-edges*.



$$\mathcal{L}(\mu X. S') = (Q, \Sigma, \delta, \mathbf{q}'_\top, q'_\perp)$$

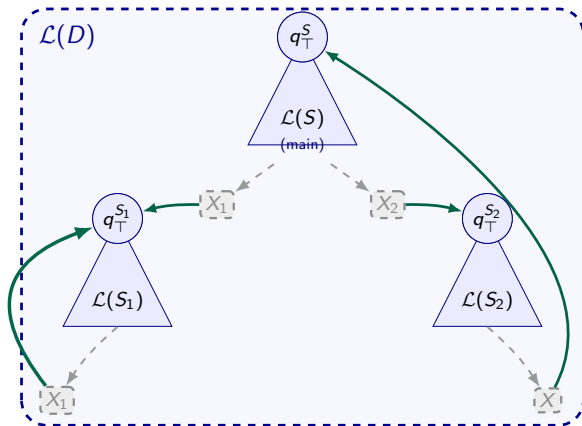
one placeholder $P_{X,j}$ per occurrence; $P_X = \{P_{X,j}\}_j$

$$Q = Q' \setminus P_X, \quad \delta = \delta' [P_{X,j} := \mathbf{q}'_\top] \text{ (back-edges)}$$

blue = carried over from $\mathcal{L}(S')$; *grey* = placeholders, removed; *green* = new back-edges.

State space: named-equations construction (Def. 3, declaration case)

One state space $\mathcal{L}(D)$ contains the main $\mathcal{L}(S)$ and a sub-session $\mathcal{L}(S_j)$ per equation $X_j=S_j$.



$$\mathcal{L}(D) = (Q, \Sigma, \delta, q_{\top}^S, q_{\perp}^S)$$

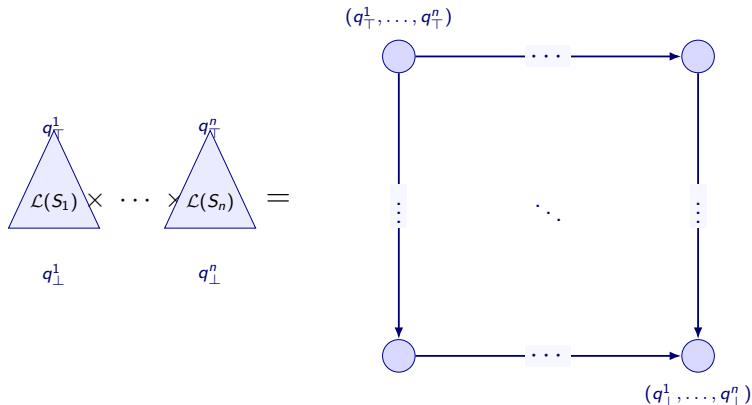
$$Q = (Q^S \cup \bigcup_j Q^{S_j}) \setminus \{p_{*,j}\}$$

δ : each $p_{*,j} \rightarrow q_{\top}^{S_j}$ (cross/back-edges)

blue carried over; grey = placeholders, removed; green = new edges.

State space: parallel construction (Def. 3, || case)

States are *tuples*; one component advances per step—the **product** $\mathcal{L}(S_1) \times \cdots \times \mathcal{L}(S_n)$.



$$\mathcal{L}(S_1 \parallel \cdots \parallel S_n) = (\prod_i Q_i, \Sigma, \delta_{\times}, (q_{\perp}^i)_i, (q_{\perp}^i)_i)$$

$\delta_{\times}$: advance one component
 $(\dots, s_i, \dots) \xrightarrow{a} (\dots, s'_i, \dots)$

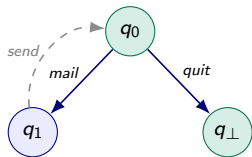
every interleaving reaches
 $(\perp_1, \dots, \perp_n)$ — the product grid
is where the **lattice** becomes
necessary.

State space: reachability order (Def. 5 · Prop. 6)

The construction yields a labelled graph. What *order* does it carry?

Def. 5 • Reachability. Write $s_1 \rightsquigarrow s_2$ (“ s_1 reaches s_2 ”) iff there is a path from s_1 to s_2 in (Q, δ) .

Prop. 6 • Preorder. $\rightsquigarrow$ is reflexive and transitive—a **preorder**. Cycles from recursion prevent antisymmetry.



cycle $q_0 \rightsquigarrow q_1 \rightsquigarrow q_0$
 $\Rightarrow$ not antisymmetric

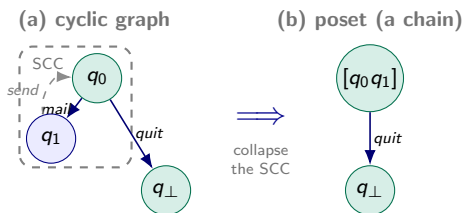
SMTP $\mu X. \&\{mail: \&\{send: X\}, quit: end\}$

Reachability orders the states—execution descends from more protocol remaining to less.

State space: SCC quotient $\rightarrow$ poset (Def. 7 · Prop. 8)

Def. 7 • SCC quotient & its order. $\mathcal{L}(S)/\equiv$ identifies mutually-reachable states (one protocol *phase* each)—the canonical antisymmetric quotient. Order $[s_1] \leq [s_2]$ iff $s_2 \rightsquigarrow s_1$: **smaller = less protocol remains**; execution descends $\top \rightarrow \perp$.

Prop. 8 • Unique extrema. For every well-formed S , $\mathcal{L}(S)$ has a unique $q_\top$ reaching all states and a unique $q_\perp$ reachable from all.



$\Rightarrow \mathcal{L}(S)/\equiv$ is a **bounded poset** (with $\top$ and $\perp$) — *but is it a lattice?*

The Reticulate Theorem

For every well-formed session type S ,
 $\mathcal{L}(S)/\equiv$ is a **bounded lattice**.

A **bounded lattice** is a poset $(L, \leq)$ with a least element $\perp$ and a greatest element $\top$ such that every pair of elements has a *meet* (greatest lower bound) $a \wedge b$ and a *join* (least upper bound) $a \vee b$.

The Reticulate Theorem: end

$$\mathcal{L}(\text{end})/\equiv = \{\bullet\} \quad (\text{one element — the base case})$$

$$\top = \perp$$

$$\wedge = \vee = \bullet$$

singleton lattice

the four ingredients:

✓ $\perp$ the one node

✓ $\top$ same node ($\top = \perp$)

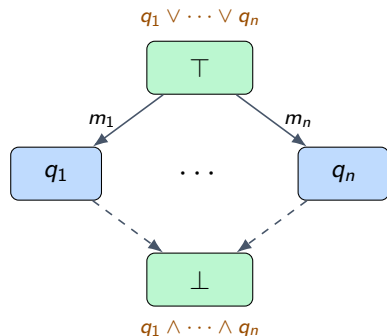
✓ $\wedge \bullet \wedge \bullet = \bullet$

✓ $\vee \bullet \vee \bullet = \bullet$

$\therefore \mathcal{L}(\text{end})/\equiv$ is a **bounded lattice** ✓

The Reticulate Theorem: branch &

If $\mathcal{L}(S_i)/\equiv$ is a bounded lattice for every $i \dots$



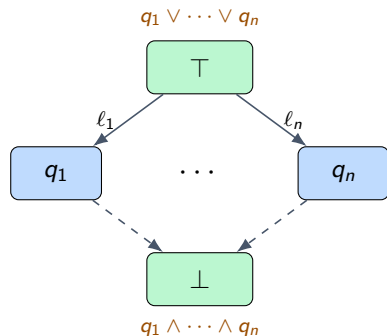
the four ingredients:

- ✓ $\perp$ shared terminal $q_\perp$
- ✓ $\top$ fresh entry q_0 (above every m_i)
- ✓ $\wedge$ branches meet at $\perp$; sub-meets in $\mathcal{L}(S_i)$
- ✓ $\vee$ branches join at q_0 ; sub-joins in $\mathcal{L}(S_i)$

$\therefore \mathcal{L}(\&\{m_i : S_i\}_{i=1..n})/\equiv$ is a **bounded lattice** ✓

The Reticulate Theorem: selection $\oplus$

If $\mathcal{L}(S_i)/\equiv$ is a bounded lattice for every $i \dots$



same diamond — polarity erased.

the four ingredients:

- ✓ $\perp$ shared terminal $q_\perp$
- ✓ $\top$ fresh entry q_0 (above every ℓ_i)
- ✓ $\wedge$ selections meet at $\perp$; sub-meets in $\mathcal{L}(S_i)$
- ✓ $\vee$ selections join at q_0 ; sub-joins in $\mathcal{L}(S_i)$

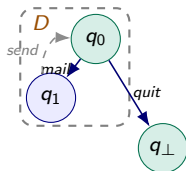
polarity matters for subtyping/duality, not reachability.

$\therefore \mathcal{L}(\oplus\{\ell_i : S_i\})/\equiv$ is a **bounded lattice** ✓

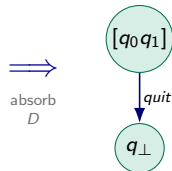
The Reticulate Theorem: recursion

If the body $\mathcal{L}(S)/\equiv$ is a bounded lattice ...

(a) cyclic body



(b) a chain



the four ingredients:

- ✓ $\perp$ survives — Termination: $\perp \notin D$
- ✓ $\top$ the absorbed class [...]
- ✓ $\wedge$ meets preserved ($L \setminus D$ down-closed)
- ✓ $\vee$ joins landing in $D \rightarrow \top$

$\therefore \mathcal{L}(\mu X. S)/\equiv$ is a **bounded lattice** ✓

The Reticulate Theorem: parallel ||

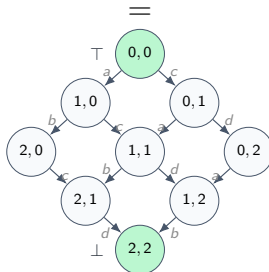
If $\mathcal{L}(S_i)/\equiv$ is a bounded lattice for each $i \dots$

$$\mathcal{L}(S_1) = \mathcal{L}(\&\{a : \&\{b : \text{end}\}\})$$



×

$$\mathcal{L}(S_2) = \mathcal{L}(\&\{c : \&\{d : \text{end}\}\})$$



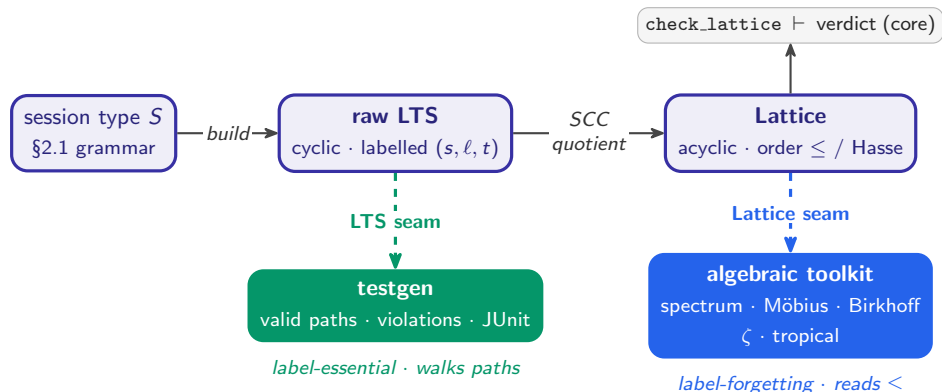
a 3×3 grid lattice.

the four ingredients:

- ✓ $\perp (\perp_1, \dots, \perp_n)$
- ✓ $\top (\top_1, \dots, \top_n)$
- ✓ $\wedge$ meets, factorwise
- ✓ $\vee$ joins, factorwise

$\therefore \mathcal{L}(S_1 \parallel \dots \parallel S_n)/\equiv = \prod_i \mathcal{L}(S_i)/\equiv$ is a **bounded lattice** ✓

Lattice Consequences: two structures, two toolsets



testgen is the *least*-derived consumer (the raw, still-cyclic LTS); the **toolkit** the *most*-derived (the quotiented order). **Each analysis has exactly one correct seam** — a test needs labels and cycles; a spectrum needs the order.

Bounded lattice	Modular	Distributive	Complemented	Rel. complemented	Uniquely compl.	Boolean + rank
Birkhoff-representable	Simple	Meet $\sqcap$	Join $\sqcup$	Complements	Duality $\mathcal{L}(S)^{op}$	Congruence quotients
Distributive reflection	Automorphism group	Join/meet-irreducibles	Birkhoff reconstr.	Rowmotion	Toggles	Order ideals
Antichains	Categorical product	Categorical coproduct	Equalizer	Coequalizer	Confluence	Height
Width	Atoms	Coatoms	Möbius function	Hall–Weisner	Möbius spectrum	Char. polynomial
Whitney numbers	Log-concavity	Zeta polynomial	Order polynomial	Linear extensions	Tutte polynomial	Spanning trees/forests
Reliability polynomial	Adjacency spectrum	Laplacian spectrum	Spectral radius	Spectral gap	Energy	Alg. conn. (Fiedler)
Cheeger	Rank valuation	FKG / log-supermod.	Monotone correlation	S/T-invariants	FCA concept lattice	Tropical spectrum

Lattice Consequences: the toolkit, on one protocol

$$S = \&\{init:\&\{open: \oplus \{OK:(\&\{read:end\} \parallel \&\{write:end\}), ERROR:end\}\}\}$$

function	what it computes	value on $\mathcal{L}(S)$
A Classify		
distributivity	distributive lattice? (M_3/N_5)	distributive
Birkhoff rep.	rebuild from join-irreducibles	iso verified (5 JI)
complements	a Boolean algebra?	no
B Operate		
automorphisms	structural symmetries	$ \text{Aut} = 2$ ($read \leftrightarrow write$)
congruences	all sound quotients	32 (not simple)
irreducibles	indecomposable blocks	5 join · 5 meet
C Measure		
height	longest run $\top \rightarrow \perp$	4
Möbius μ	inclusion–exclusion	$\max \mu =1$, 8 intervals
char. + Whitney	rank statistics	deg 4, log-concave
Tutte	spanning structure $\rightarrow$ reliability	4 trees, $\chi(2)=2$
spectral	connectivity · energy	$\lambda_2=0.28$, energy 8.1

The numbers

The algebraic census

108

benchmark protocols

(networking, DB, AI, distributed)

93

distributive (86%)

(no forbidden N_5 or M_3)

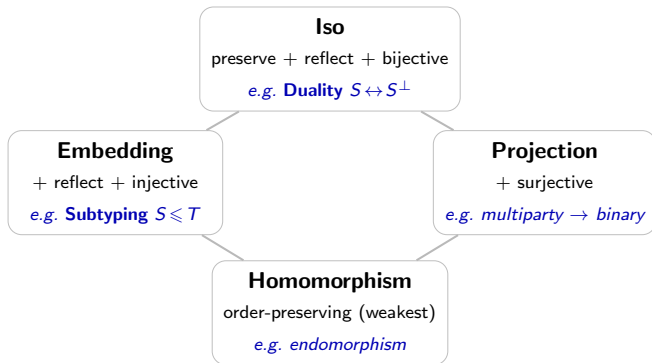
15

non-distributive (14%)

(13 with N_5 , 2 with M_3)

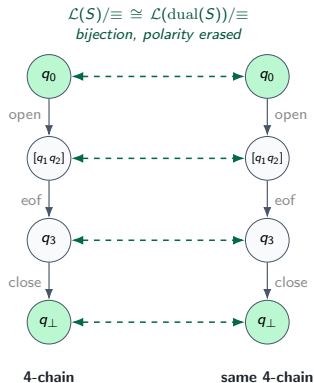
All 108 form bounded lattices. **Reticulate Theorem holds universally on the suite.**

Lattice Consequences: Morphism Hierarchy



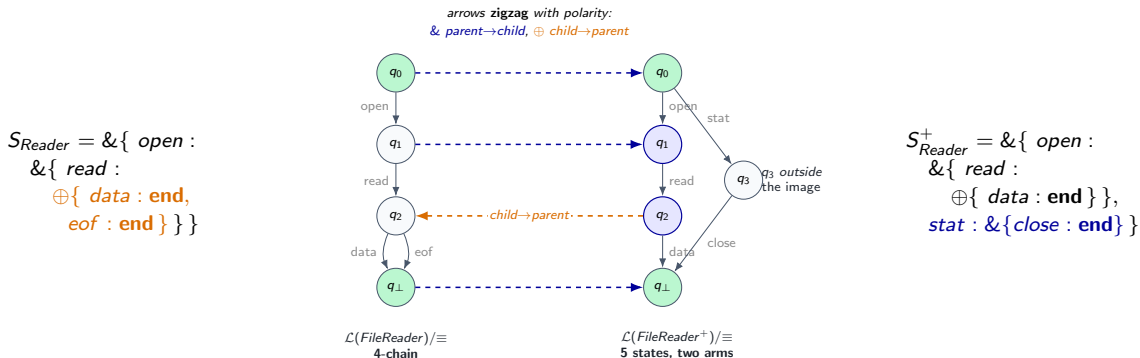
*Is the **bare lattice** — as an abstract interpretation — enough to characterise **duality** and **subtyping**?*

$$S_{\text{Reader}} = \&\{ \text{open} : \\ \mu X. \&\{ \text{read} : \\ \oplus\{ \text{data} : X, \\ \text{eof} : \&\{ \text{close} : \mathbf{end} \} \} \} \}$$

$$\text{dual}(S_{\text{Reader}}) = \oplus\{ \text{open} : \\ \mu X. \oplus\{ \text{read} : \\ \&\{ \text{data} : X, \\ \text{eof} : \oplus\{ \text{close} : \mathbf{end} \} \} \} \}$$


Given S and $\text{dual}(S) \Rightarrow \mathcal{L}(S)/\equiv \cong \mathcal{L}(\text{dual}(S))/\equiv$

Lattice Consequences: Subtyping (FileReader $\leq$ a subtype, {, Not Sound Not Faithful

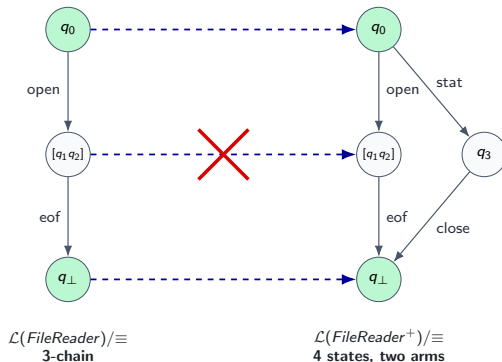


1. Branch width. $S \leq_{GH} T \Rightarrow \mathcal{L}(T)/\equiv$ embeds into $\mathcal{L}(S)/\equiv$.
2. Selection width. $S \leq_{GH} T \Rightarrow \mathcal{L}(S)/\equiv$ embeds into $\mathcal{L}(T)/\equiv$.

Lattice Consequences: Subtyping (recursive read-loop, §5)

Not Sound

Not Faithful

$$S_{\text{Reader}} = \&\{ \text{open} : \\ \mu X. \&\{ \text{read} : \\ \oplus\{ \text{data} : X, \\ \text{eof} : \text{end}, \\ \text{other} : X \} \} \}$$


$$S_{\text{Reader}}^+ = \&\{ \text{open} : \\ \mu X. \&\{ \text{read} : \\ \oplus\{ \text{data} : X, \\ \text{eof} : \text{end} \} \}, \\ \text{stat} : \&\{ \text{close} : \text{end} \} \}$$

Recursion breaks soundness. the SCC quotient **collapses the read-loop** into one node $[q_1 q_2]$, destroying the *continuity* of its continuations—the ordered branch/selection chain that subtyping pairs up—so *no* embedding survives the quotient, sound or faithful.

Lattice Consequences: the way to faithfulness

The bare lattice is **sound** but **not faithful**:
the *SCC quotient* forgets structure, so distinct types can share one lattice.

what is lost	witness	bare $\mathcal{L}(S)$	faithful $\mathcal{L}(\cdot)^\dagger$
width	$\&\{a\}$ vs $\&\{a, b\}$	distinct	distinct
labels	$\&\{a\}$ vs $\&\{b\}$	collapses	keep δ
polarity	$\&\{a\}$ vs $\oplus\{a\}$	collapses	tag $\&/\oplus$
recursion	$\mu X. \&\{r: \oplus\{d:X\}\}$	continuity lost	unfold μ once

Lattice Consequences: two abstract interpretations

Reticulate Theorem

the **bare lattice** — **sound** (proved on 108)

ICE 2026

$$S \Rightarrow \mathcal{L}(S) \Rightarrow \mathcal{L}(S)/\equiv$$

Conjecture

the **faithful lattice** — **sound** + **faithful** + **realisable**

???

$$S \Leftrightarrow \mathcal{L}(S) \Leftrightarrow \mathcal{L}(S)^\dagger$$

Conjecture (faithful refinement). *Change the technique:* replace the **SCC quotient** by a **coinductive** one — unfold each recursion once and identify states up to **labelled bisimulation**. The refined reticulate $\mathcal{L}(S)^\dagger$ is again a bounded lattice and characterises Gay–Hole subtyping *exactly*:

1. **Branch width:** $S \leq_{GH} T \iff \mathcal{L}(T)^\dagger \hookrightarrow \mathcal{L}(S)^\dagger$
2. **Selection width:** $S \leq_{GH} T \iff \mathcal{L}(S)^\dagger \hookrightarrow \mathcal{L}(T)^\dagger$

Questions?

Scan to explore the reticulate live



reticulate.bica-tools.org